Recap through exercise --

X x_1 : {Block, Jeevan, Cinema, Old, Agra}
x_2 : {House, Jeevan, Vihar, Delhi}

Y x_3 : {Block, Jeevan, Cinema, Agra}
x_4 : {Building, Ravi, Vihar, New, Delhi}
x_5 : {Block, Jeevan, Vihar, New, Delhi}
x_6 : {Mandir, Ravi, Vihar, Delhi}

Inverted Index

x_1: {Block, Jeevan, Cinema, Old, Agra}

What will be the candidate set?

Candidate set: {3, 5}+{3, 5}+{3}+{3}

X x_1 : {Block, Jeevan, Cinema, Old, Agra}
x_2 : {House, Jeevan, Vihar, Delhi}

Y x_3 : {Block, Jeevan, Cinema, Agra}
x_4 : {Building, Ravi, Vihar, New, Delhi}
x_5 : {Block, Jeevan, Vihar, New, Delhi}
x_6 : {Mandir, Ravi, Vihar, Delhi}

IDX(S)

Token	DID
Agra	3
Block	3, 5
Cinema	3
Delhi	4, 5, 6
Jeevan	3, 5
Mandir	6
New	4, 5
Ravi	4, 6
Vihar	4, 5, 6

Size Filtering

X x_1 : {Block, Jeevan, Cinema, Old, Agra}
x_2 : {House, Jeevan, Vihar, Delhi}

Y x_3 : {Block, Jeevan, Cinema, Agra}
x_4 : {Building, Ravi, Vihar, New, Delhi}
x_5 : {Block, Jeevan, Vihar, New, Delhi}
x_6 : {Mandir, Ravi, Vihar, Delhi}

Given a string xεX, only strings y such that $|x|/t \geq |y| \geq |x|*t$

Consider x_1={Block, Jeevan, Cinema, Old, Agra}, suppose t=0.9

What will be the candidate set?

- If yεY matches x, we must have
  - $5/0.9 = 5.5 \geq |y| \geq 5* 0.9 = 4.5$
  - Candidate set C= x_4, x_5

IDX(S)

Token	DID
Agra	3
Block	3, 4, 5
Cinema	3
Delhi	4, 5, 6
Jeevan	3, 5
Mandir	6
New	4, 5
Ravi	4, 6
Vihar	4, 5, 6

Prefix Filtering

- if $|x \cap y| \geq k$ ⇒ any subset $x' \subseteq x$ of size at least $|x| - (k - 1)$ must overlap y
- Consider matching using $O(x,y) \geq 3$
- x_1={Block, Jeevan, Cinema, Old, Agra}
- What would be the candidate set?

X x_1 : {Block, Jeevan, Cinema, Old, Agra}
x_2 : {House, Jeevan, Vihar, Delhi}

Y x_3 : {Block, Jeevan, Cinema, Agra}
x_4 : {Building, Ravi, Vihar, New, Delhi}
x_5 : {Block, Jeevan, Vihar, New, Delhi}
x_6 : {Mandir, Ravi, Vihar, Delhi}

- Prefix x_1' = {Block, Jeevan, Cinema},
- Use inverted index to find the candidate set C={x_3, x_5} which contain at least one token in x_1'.

Prefix-Inverted Index

X
x_1 : {Block, Jeevan, Cinema, Old, Agra}
x_2 : {House, Jeevan, Vihar, Delhi}

Y
x_3 : {Block, Jeevan, Cinema, Agra}
x_4 : {Building, Ravi, Vihar, New, Delhi}
x_5 : {Block, Jeevan, Vihar, New, Delhi}
x_6 : {Mandir, Ravi, Vihar, Delhi}

Order:
Building < House < Mandir < Old < Agra < Cinema < New < Ravi < Block < Delhi < Jeevan < Vihar

Token	ID List	TF	Order
Agra	1, 3	2	5
Block	1, 3, 5	3	9
Building	4	1	1
Cinema	1, 3	2	6
Delhi	2, 4, 5, 6	4	10
House	2	1	2
Jeevan	1, 2, 3, 5	4	11
Mandir	6	1	3
New	4, 5	2	7
Old	1	1	4
Ravi	4, 6	2	8
Vihar	2, 4, 5, 6	4	12

Order: x_1 : {Block, Jeevan, Cinema, Old, Agra}

Building < House < Mandir < Old < Agra < Cinema < New < Ravi < Block < Delhi < Jeevan < Vihar

Algorithm

- reorder terms in each x ε X and y ε Y in increasing order of their frequencies
- for each y εY , create y', the prefix of size |y| - (k – 1) of y
- build an inverted index over all prefixes y'
- for each x ε X, create x', the prefix of size |x| - (k – 1) of x, then use above index to find all y such that x' overlaps with y'

$O(x,y) \geq 3$ Prefix(x)=|x|-(3-1)=|x|-2

What would be the candidate set?

X x_1 : {Old, Agra, Cinema, Block, Jeevan}
x_2 : {House, Delhi, Jeevan, Vihar}

Y x_3 : {Cinema, Agra, Block, Jeevan}
x_4 : {Building, New, Ravi, Delhi, Vihar}
x_5 : {New, Block, Delhi, Jeevan, Vihar,}
x_6 : {Mandir, Ravi, Delhi, Vihar}

Prefix Inverted Index for Y

Token	DID
Agra	3
Cinema	3
Delhi	5
Mandir	6
New	4, 5
Ravi	4, 6

Candidate Set C for x_1={Old, Agra, Cinema, Block, Jeevan} is {x_3}.

Position Index

x_1={Old, Agra, Cinema, ~~Block, Jeevan~~}
x_3 : {Cinema, Agra,~~ Block, Jeevan~~}

O(x,y) >4

Estimation of max overlap = overlap in prefixes + min of unseen tokens = 2 + min(2, 2) = 4 > k (false)

 No need to compute sim(x_1, x_3) !

Cleansing Composite Attributes with Ripple Down Rules

(Composite Entity (Description) Normalization (and Cleansing) into DB Schema)

BRIDGE FROM PREVIOUS TOPIC

From matching records to cleansing attributes

PREVIOUS LECTURE: FIND CANDIDATES

X: {Block, Jeevan, Cinema, Old, Agra}
Y: x3={Block, Jeevan, Cinema, Agra}

Use token inverted indexes, size bounds, prefix and position filtering to reduce pair comparisons.

TODAY: RESOLVE A TTRIBUTE MEANINGS

Find candidate duplicates first; then parse each messy address into House, Block, Road, Area, City and PIN.

I-344, Sarojni Ngr, N Delhi, 23
344, Block I, Sarojini Nagar, New Delhi, 110023

Matching does NOT determine field boundaries or canonical names.

Question: could two addresses have high token overlap yet be different houses?
YES — field semantics matter.

A concrete integration problem

PRODUCT FEEDS

“SPEED STK FRESH”
“DRY IDEA GEL POWDR FRSH 2.25OZ A P”
“HP 15s 8GB RAM 512GB SSD”

POSTAL ADDRESS FEEDS

“I-344 Sarojni Ngr N Delhi 23”
“344 Block I Sarojini Nagar New Delhi 110023”

ELEMENTISATION / PARSING

Identify raw spans and assign fields.

“FRSH” → [Variant: FRSH]
“2.25OZ” → [Size: 2.25OZ]
“N Delhi” → [City: N Delhi]

STANDARDISATION + VALIDATION

Convert extracted spans to canonical values.

FRSH → Fresh
2.25OZ → 2.25 oz
N Delhi → New Delhi

Validate consistency; preserve evidence.



Without normalization: joins miss, deduplication fails, analytics group aliases as distinct values.

Understanding dirty data

Why dictionaries alone do not solve the problem

Sources of difficulty: six failure modes

Failure	Example	Why hard
Abbreviation	STK; FRSH; N Delhi	Many aliases, domain-dependent
Ambiguous token	St John St	Saint vs Street
Omitted field	SPEED STK FRESH	Size absent; cannot hallucinate
Token order	344 Block I / Block I-344	Flexible grammar
Typos / spacing	Sarojni	Alias and segmentation issues
Overloaded number	8GB RAM; 512GB SSD	Same unit, different semantics

Product descriptions → extracted fields

Raw source	Parsed concept 1	Parsed concept 2
SPEED STK FRESH	Brand / line: Speed Stick	Variant: Fresh; size unknown
DRY IDEA GEL POWDR FRSH 2.25OZ A P	Brand: Dry Idea; form: Gel/Powder	Size: 2.25 oz; AP: antiperspirant
SURE 2.6OZ UNSCNT INV SLD	Brand: Sure; form: Solid	Unscented; Invisible; 2.6 oz
GILLETTE CLWV GEL TWNPK DEO 6OZ	Brand: Gillette; form: Gel	Twin pack; deodorant; 6 oz
CVS MEN DEOD REG 325ml	Brand: CVS; gender: Men	Deodorant; Regular; 325 mL

Treat ambiguous tokens as unresolved until context/dictionary support exists.

Postal addresses: Indian variants

Variant spellings, partial postal codes, and non-identical field boundaries

THREE RECORDS THAT MAY REFER TO ONE LOCATION

1. I-344 | Sarojini Nagar | N Delhi | 23
2. 344 Block I | Sarojni Ngr | New Delhi | 110023
3. 344 Block I Near Navyug School S Ngr NDelhi23

EXPECTED PROCESSING

Extract block/door + landmark + area + city + postal code → standardise aliases → validate locality, PIN and house identity.

Do not assume “23” uniquely determines 110023 without geographic evidence.

Why address standardisation fails: three original cases

SPELLING + ABBREVIATIONS

I-344 | Sarojini Nagar | N Delhi | 23

344 Block I | Sarojni Ngr | New Delhi | 110023

MULTIPLE FIELD BOUNDARIES

13B | Link Road | Versova | Mumbai

Block B-13 | Bandra Versova Link Rd. | Versova | Mumbai

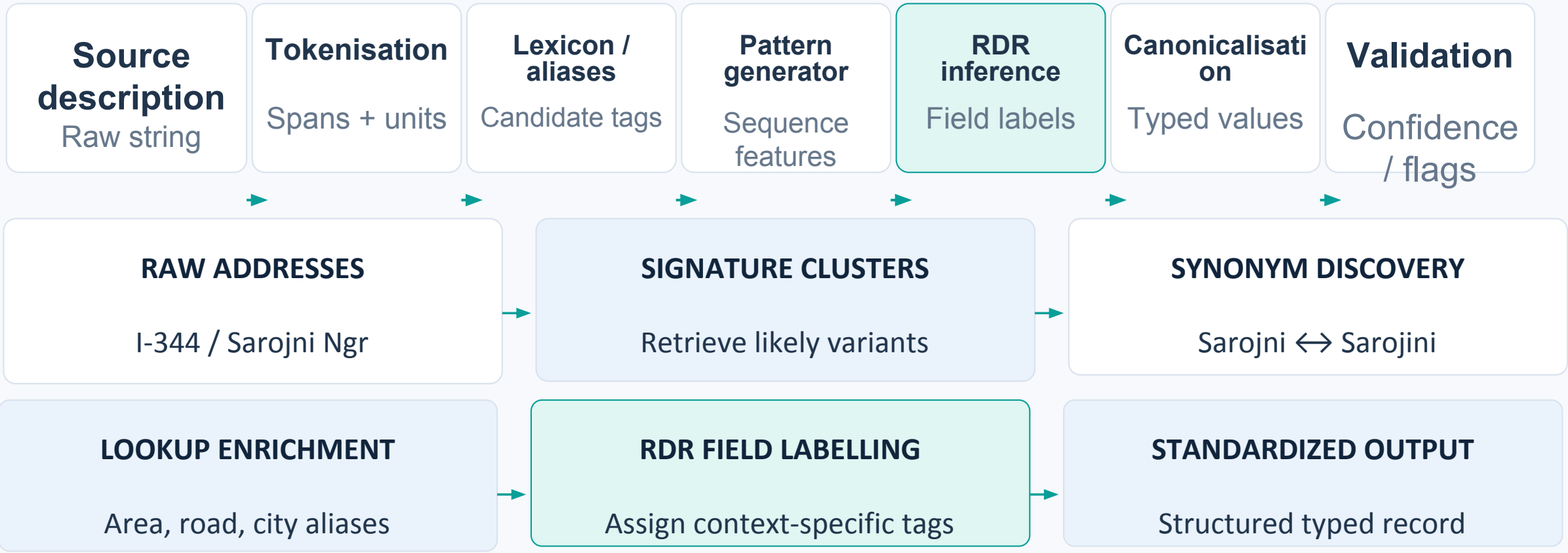
OMITTED / REORDERED FIELDS

123 | IIITD Campus | Okhla Phase 3 | New Delhi | 20

123 Block-C | IIIT Delhi | Industrial Area | Okhala Phase III | New Delhi | 110020

These are candidate variants, NOT automatically identical locations. Normalize → validate → entity-match.

Pipeline: where each component fits



Supporting resources:

- **curated dictionaries**
- **cornerstone cases**
- **Data provenance**

locale rules
regression tests

Clustering Based Approach

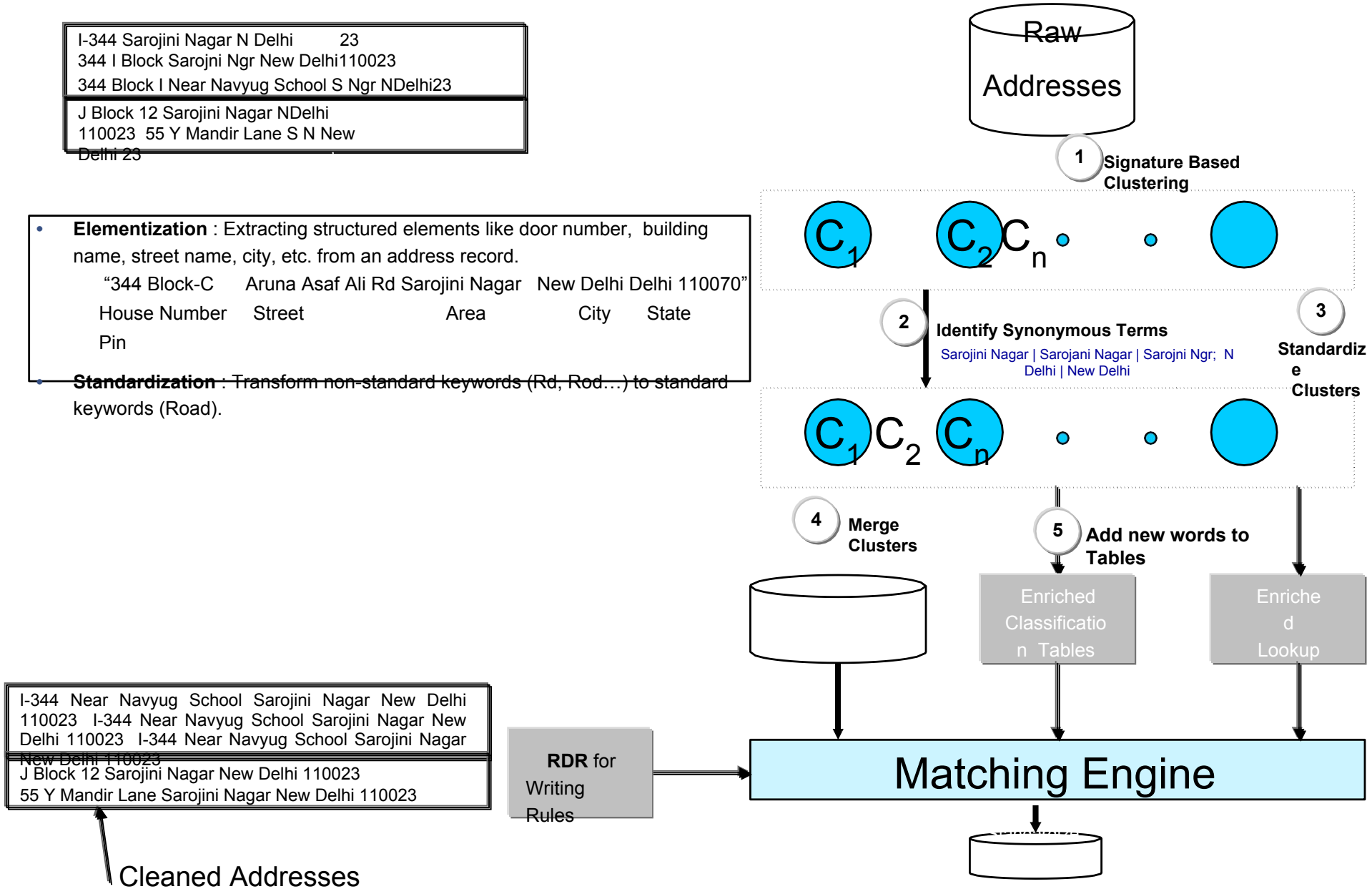
I-344 Sarojini Nagar N Delhi 23 344 I Block Sarojni Ngr New Delhi110023 344 Block I Near Navyug School S Ngr NDelhi23
J Block 12 Sarojini Nagar NDelhi 110023 55 Y Mandir Lane S N New Delhi 23

- **Elementization** : Extracting structured elements like door number, building name, street name, city, etc. from an address record.

  "344 Block-C Aruna Asaf Ali Rd Sarojini Nagar New Delhi Delhi 110070"

  House Number Street Area City State Pin

- **Standardization** : Transform non-standard keywords (Rd, Rod…) to standard keywords (Road).



Home Assignment – 2 marks (Due Date: October 19, 2026)

1. Select the addresses from any metro city/state from the electoral data (from Election Commission website or any other source) [the cardinality of Address column should not be less than 10M], and build a look-up dictionary for all variants of Road Type (Rd, St, Street, Road, Path, Marg, Lane,…).

2. Marks will be given based on your prompt, data size, and running the reviewed (generated) code on the address data, and building the look-up table/dictionary. You need to highlight where you have made the changes in the LLM generated code.

Lexicons and pattern mining

From messy tokens to robust candidate predicates

Feature extraction before RDR

Surface + shape

H171 → alpha+digits
110020 → six digits
2.25OZ → number+unit

Dictionary matches

Road / Rd / Marg
Apartments / Towers
New Delhi / Bengaluru

Contextual signals

Previous token = “Flat”
Next token = “RAM”
Country = India

Higher-order signals

Detected City=Delhi
Product category=laptop
Street begins after house no.

Important: longest-phrase dictionary lookup before single-token tagging helps preserve “New Delhi”, “Dry Idea”, “Saki Naka”.

Your sub-pattern notation

SYMBOL DICTIONARY

B = building-type marker
T = street-type marker
A = area / town
C = city / district
S = state
P = PIN
^ = numeric token
+ = currently unknown token

ENCODE DESCRIPTIONS

127 Mahima Towers Mahatma Gandhi Road Calcutta
→ ^+B++TC

344 Block-C Aruna Asaf Ali Rd Sarojini Nagar New Delhi Delhi 110070
→ symbolic stream; multiword recognition and tokenisation policy must be specified.

Purpose: mine recurring local contexts.

Sub-pattern Mining

Input: Indian Address Text file, Database Schema, Classification and Lookup Tables, Input Pattern (P)
Output: Sub-patterns that will assist in writing the rules

Approach:

- Assume we would like to normalize each address(address database schema) into

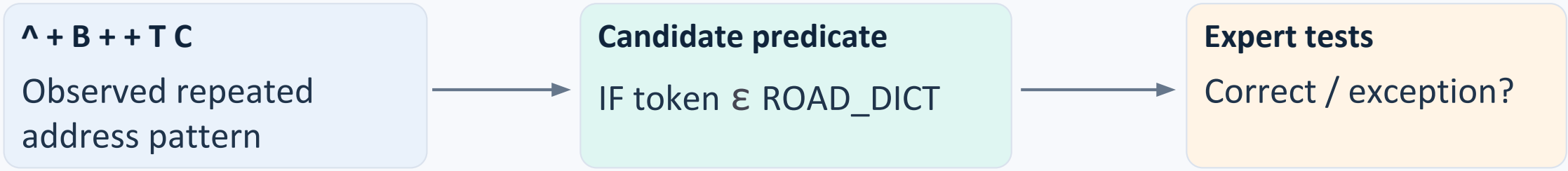
  ADDRESS(Floor#, House#, Building_Name, Landmark, Street_Name, Street_Type, Area, City, State, PIN) B
  T A C S P

- Convert each address into a pattern using Classification and Lookup Tables
  - *Street, Road, Avenue, Marg,* Nagar, Phase are street type markers represented by T
  - *Building, Towers, Bunglow* are building type markers represented by B
  - Tables for Area/Town (A), City/District (C), State (S), PIN (P)
  - \+ represents unknown word, ^ represents numeric token
  - *127 Mahima Towers Mahatma Gandhi Road Calcutta* → *^+B++TC*
  - 344 Block-C Aruna Asaf Ali Rd Sarojini Nagar New Delhi Delhi 110070 → ^++++T+ACSP
  - 123 IIIT Delhi Okhala Phase-3 New Delhi 110020 → ^+SACP

- Find a sub-pattern for writing the rules

- Rules size will be very large… so need some method to manage such large rules.

From pattern frequency to a decision rule



Why not directly replace the pattern with a rule?
Because “Road” could be part of a landmark, a building name or a street type depending on context.

Ripple Down Rules

Maintain knowledge by appending contextual exceptions

Ripple Down Rules (RDR) – Managing Rules

- Knowledge acquisition methodology: the human expert's knowledge is acquired based on the current context and is added incrementally
- Rule Tree: In its simplest version, each node is a rule and has two child nodes (except, else) depending on whether the rule is satisfied or not
- Rules are added or modified when the expert system incorrectly classifies a case or fails to classify a case
- The case which prompted the addition of this rule and the set of cases this rule classified correctly are stored for reference.

Ripple-Down Rules

if *condition* **then** *conclusion* [**because** case] **except**
    if ....
else if ...

For example:

if (a and b) then c except	a and b => c
if d then e	a and b and d => e
else if (f and g) then h	f and g => h

Default rule:
    If true then normal
Rule 1: when an exception If (a and b) then c is added
    If true then normal except
        If (a and b) then c
Rule 2: when an exception If (f and g) then h is added
    If true then normal except
        If (a and b) then c
        else if (f and g) then h
Rule 3: when another exception If (a, b and d) then e is added
    If true then normal except
        If (a and b) then c except
            if d then e
        else if (f and g) then h

⇒ If a rule fires, but produces an incorrect conclusion, we add an **except** branch

⇒ => If a rule fails to fire, we add an **else** branch

Ordinary **if-then-else** statement

```
if (a and b) then
    if d then e
    else c
else if (f and g) then h
```

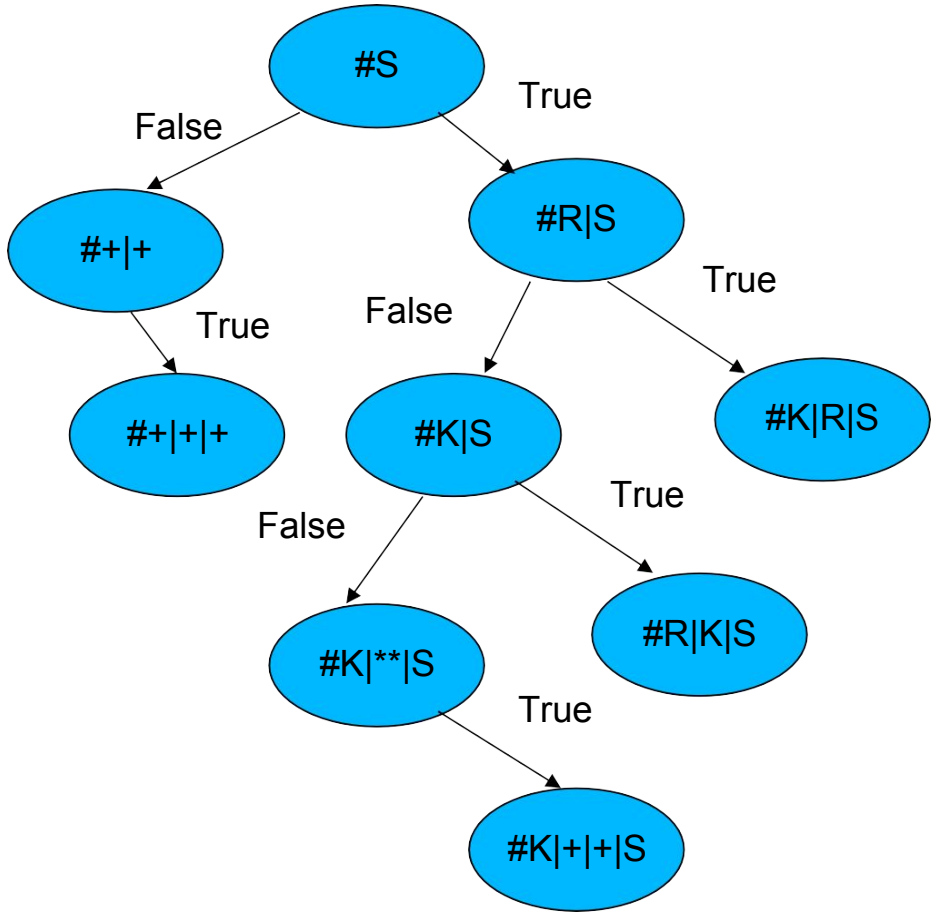
Knowledge acquisition process in RDR

If the rule in a node is true, the rule in the node connected by TRUE branch is also tested.

The conclusion of the parent is returned only if the rule on the TRUE branch does not satisfy the case.

If a rule is not satisfied for a case, the child rule with a FALSE link is tested.

If the case satisfies the child rule, its conclusion takes precedence over the conclusion of the parent rule.

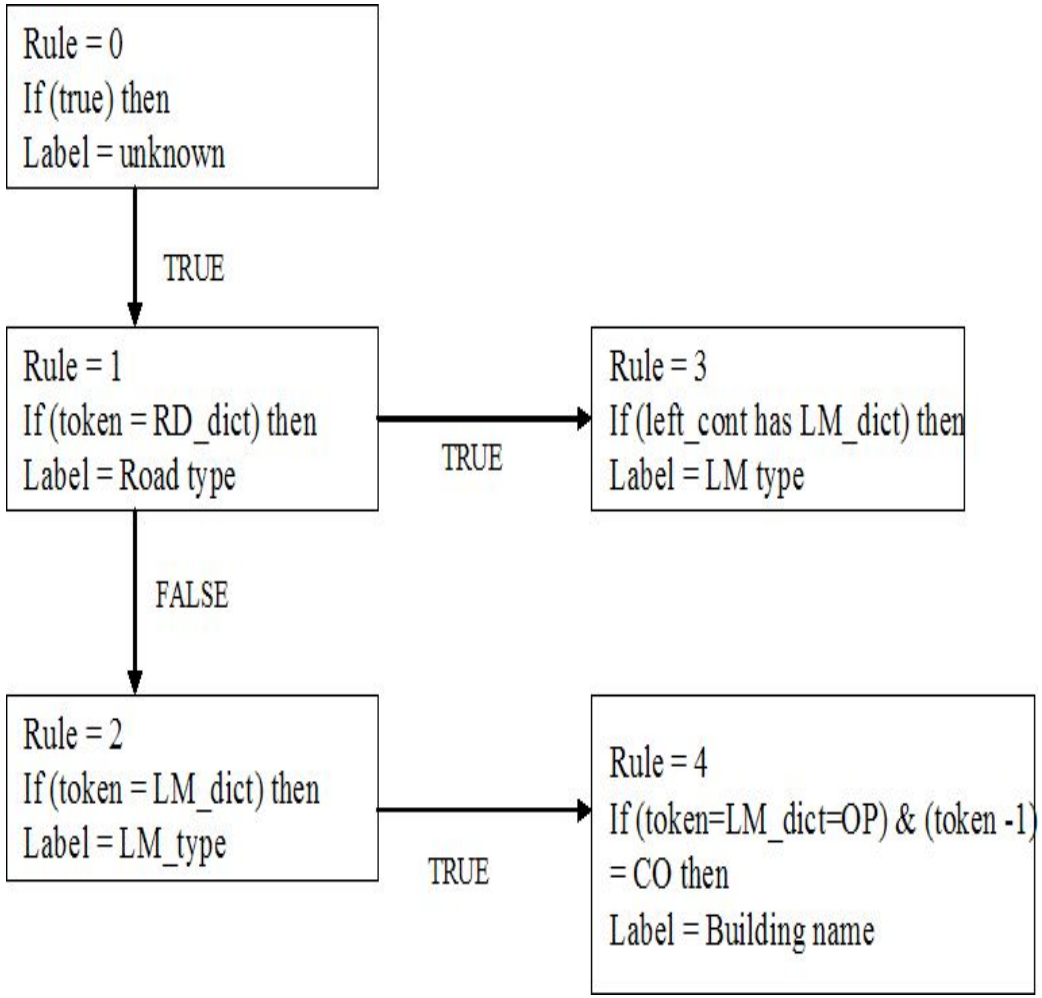


Rule organization in RDR framework

1. For Rule 1, if the current token is in road dictionary, do not declare it as a road type immediately, but perform step 2. If the current token is not in road dictionary, go to Step 3.

2. Check if the left context of this token has a landmark type. If yes, declare the token as part of landmark rather than as a road type, else road type.

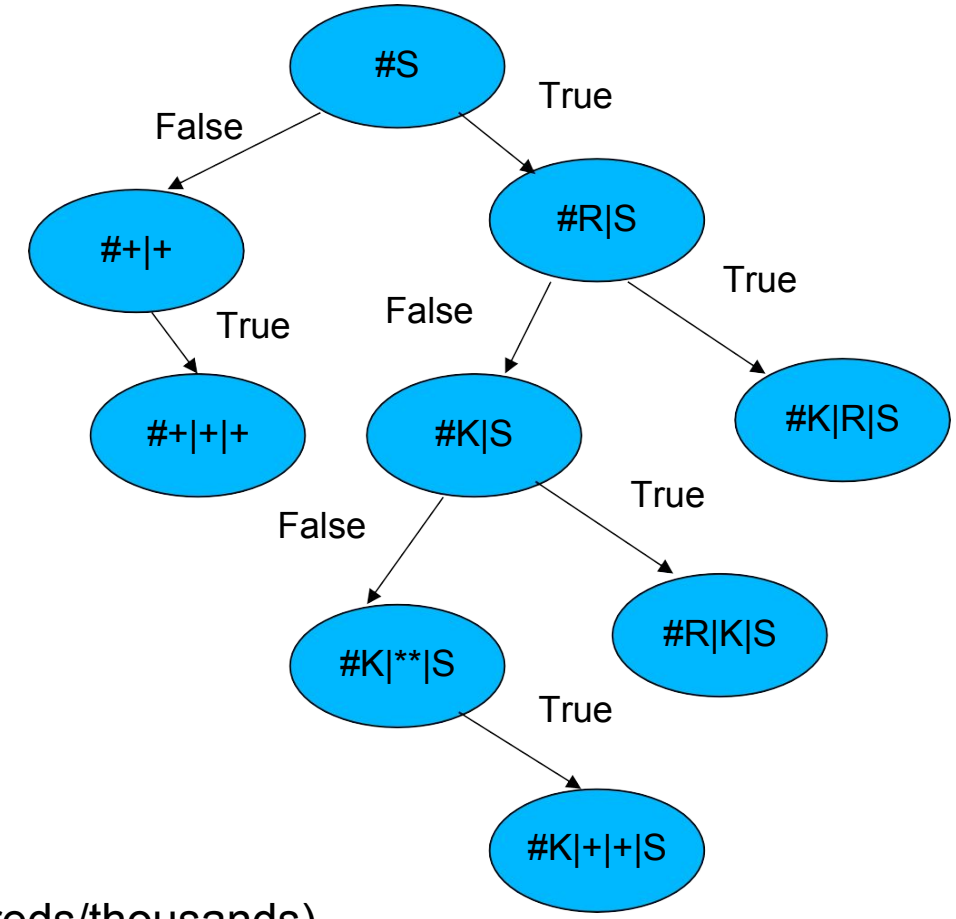
3. Check Rule 2, .. and so on.



RDR Tree – a summary

Each Node will have Three steps

- A True Branch to look for more specific or elaborate set of actions you want to take
- If a more specific case is not executed then existing generic action is taken
- A False case to look at alternate patterns



Each Node entered is being traced and noted in Rule order Execution

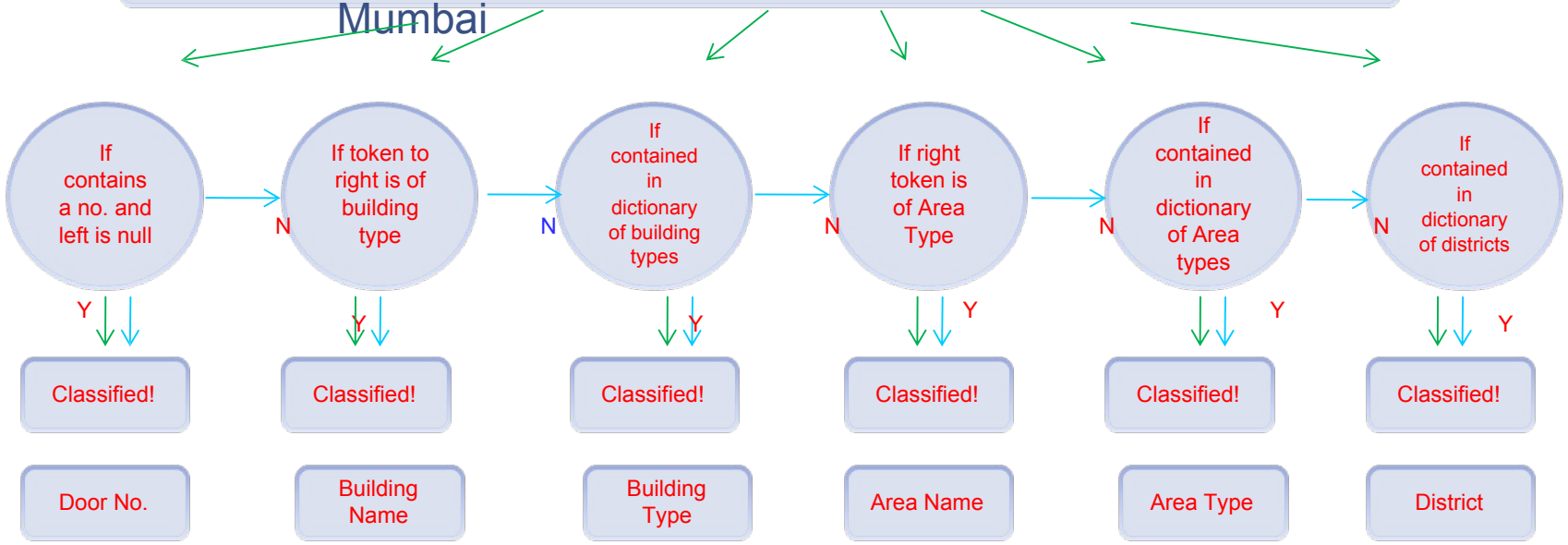
- This will help trace for a given address which particular case is breaking
- It guides the person to add a new rule to its false branch

Expect large number of rules (in hundreds/thousands)

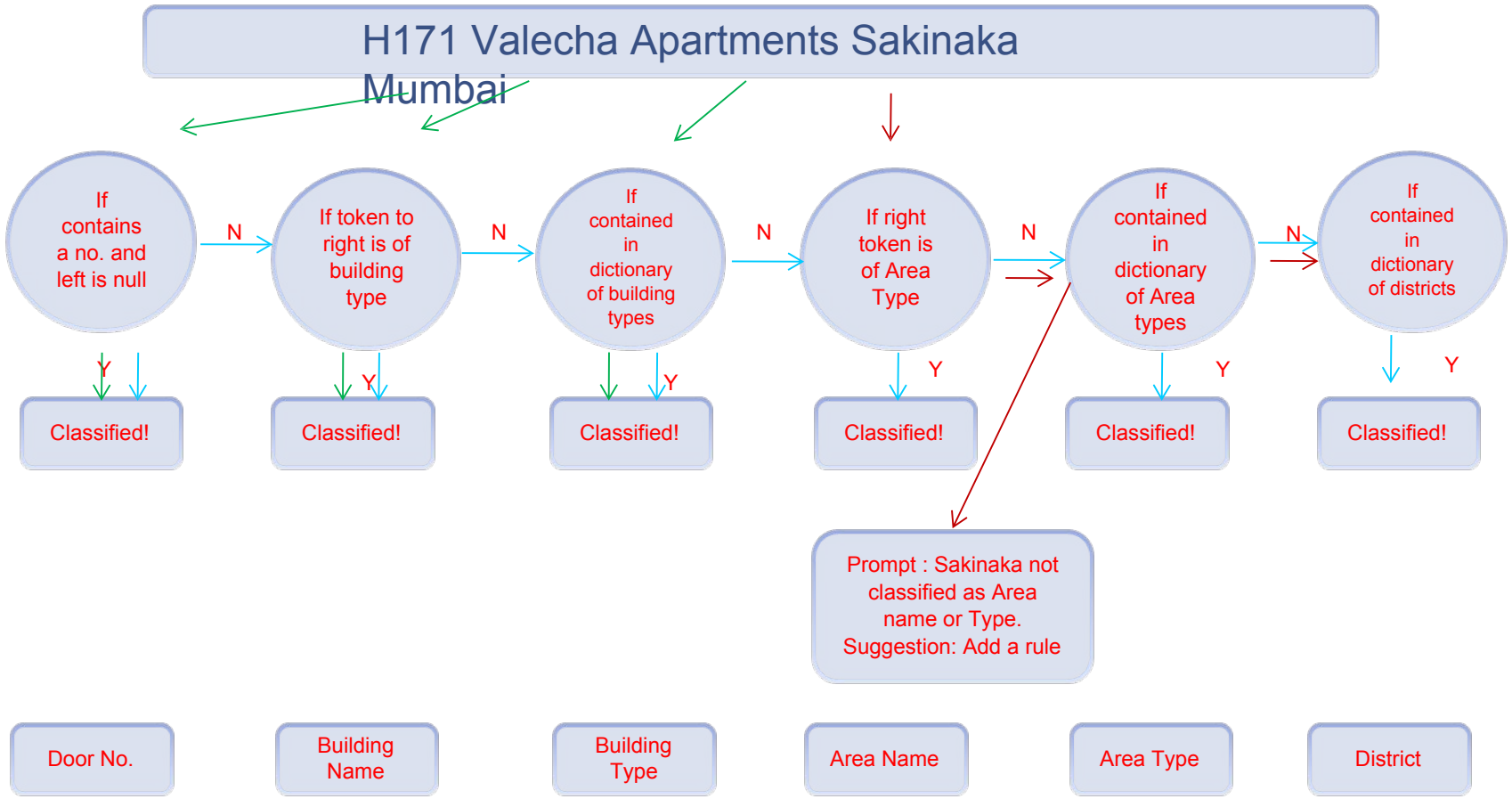
- Rules are organized in the form of a taxonomy
- Order rules within each node based on the generality of a rule

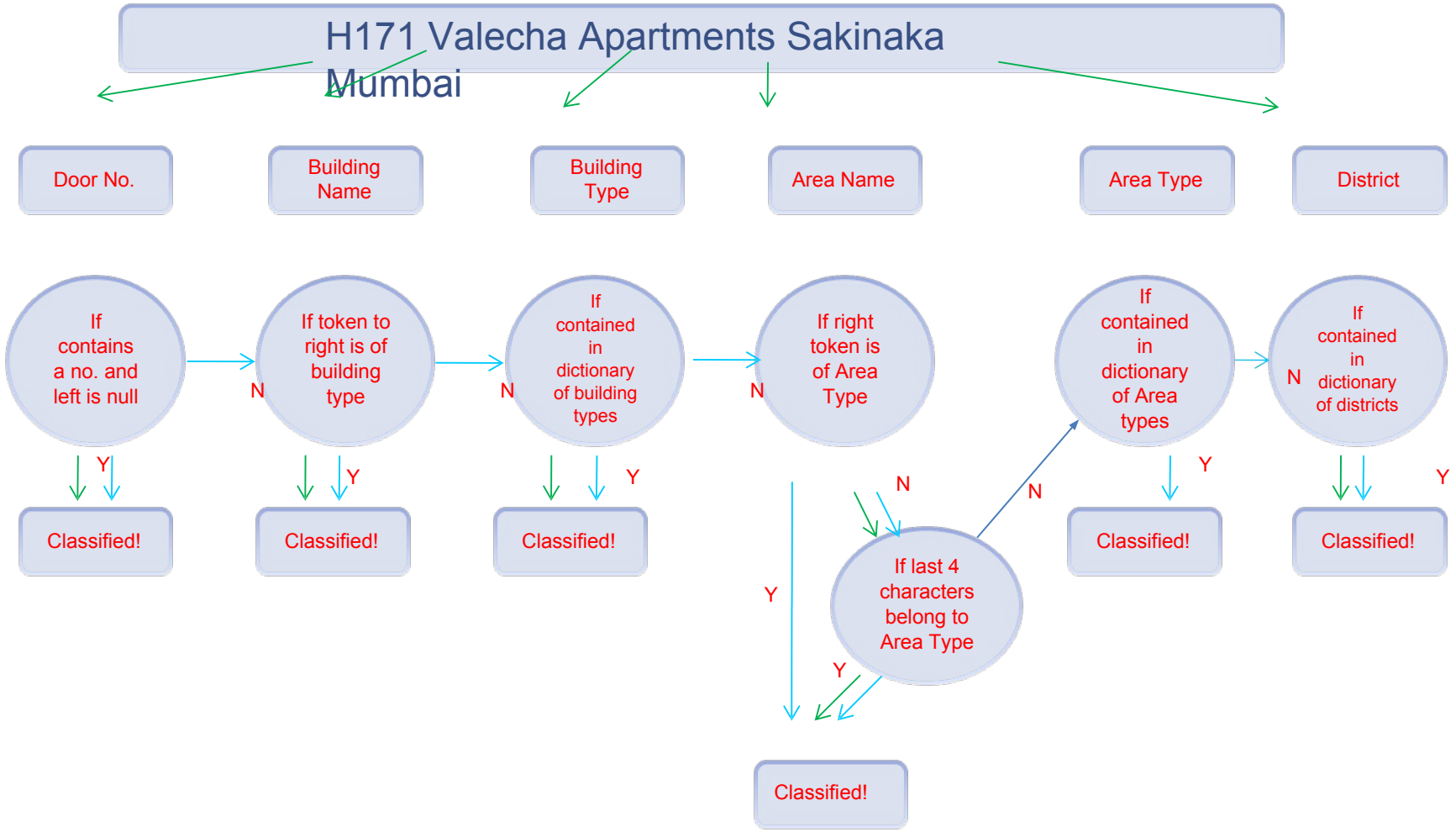
Annotate rules with examples to make it clear how they will work

H171 Valecha Apartments Saki Naka Mumbai



What if query = “H171 Valecha Apartments Sakinaka Mumbai”


H171 Valecha Apartments Sakinaka Mumbai
If contains a no. and left is null
N
If token to right is of building type
N
If contained in dictionary of building types
N
If right token is of Area Type
N
If contained in dictionary of Area types
N
If contained in dictionary of districts
Y
Classified!
Y
Classified!
Y
Classified!
Y
Classified!
Y
Classified!
Y
Classified!
Prompt : Sakinaka not classified as Area name or Type. Suggestion: Add a rule
Door No.
Building Name
Building Type
Area Name
Area Type
District


H171 Valecha Apartments Sakinaka Mumbai
Door No.
Building Name
Building Type
Area Name
Area Type
District
If contains a no. and left is null
If token to right is of building type
If contained in dictionary of building types
If right token is of Area Type
If last 4 characters belong to Area Type
If contained in dictionary of Area types
If contained in dictionary of districts
Y
N
Classified!